

Prueba - Obtención y preparación de datos

En esta prueba validaremos nuestros conocimientos adquiridos durante el módulo.

Lee todo el documento antes de comenzar el desarrollo individual, para asegurarte de tener el máximo de puntaje y enfocar bien los esfuerzos.

Descripción

El departamento de Recursos Humanos de una empresa necesita analizar la distribución de salarios en sus diferentes departamentos y ubicaciones. La información se encuentra en dos conjuntos de datos separados: uno con los datos de los empleados y otro con la información de los departamentos. Tu misión es integrar, agrupar y reestructurar estos datos para presentar un informe claro que compare los salarios promedio.

Resultado de Aprendizaje a Evaluar:

Aplicar técnicas de unión, agrupamiento y pivoteo de datos utilizando la librería pandas para el reacomodo de la información para la resolución de un problema.

Instrucciones

Copia el siguiente código para crear los dos DataFrames de Pandas que necesitarás para el desafío.

```
Python
import pandas as pd

# DataFrame con información de empleados
data_empleados = {
    'id_empleado': [101, 102, 103, 104, 105, 106, 107, 108],
    'nombre': ['Ana', 'Luis', 'Sofía', 'Carlos', 'Elena', 'Pedro', 'Laura',
'David'],
    'salario': [55000, 58000, 72000, 48000, 75000, 85000, 62000, 88000],
    'id_departamento': ['V_01', 'V_01', 'T_01', 'M_01', 'T_01', 'D_01',
'M_01', 'D_01']
}
df_empleados = pd.DataFrame(data_empleados)

# DataFrame con información de departamentos
data_departamentos = {
    'id_departamento': ['V_01', 'T_01', 'M_01', 'D_01'],
    'nombre_depto': ['Ventas', 'Tecnología', 'Marketing', 'Dirección'],
}
```

```
    'ubicacion': ['Norte', 'Sur', 'Norte', 'Norte']  
}  
df_departamentos = pd.DataFrame(data_departamentos)
```

Requerimientos

1. Combinación de Datos:
 - Une los dos DataFrames en uno solo, llamado `df_completo`. Asegúrate de que cada empleado quede asociado con el nombre y la ubicación de su departamento correspondiente.
2. Agrupamiento y Agregación:
 - Usando `df_completo`, agrupa los datos para calcular el salario promedio por cada `nombre_depto` y `ubicacion`.
 - Muestra el resultado de esta agrupación.
3. Pivoteo de Datos:
 - Reestructura el resultado del paso anterior para crear una tabla pivote. El índice de la tabla debe ser el `nombre_depto`, las columnas deben ser la `ubicacion`, y los valores deben ser el salario promedio calculado.
 - Muestra la tabla pivote final. El resultado debería facilitar la comparación directa de salarios entre ubicaciones para un mismo departamento.



¡Mucho éxito!

Estudiante: Valeria Fariña

Programa: Especialización en Fundamentos de Ciencia de Datos

Institución: IGT Chile / Corporación Nueva Ciaspo (SENCE - Talento Digital para Chile)

URL del Repositorio:

https://github.com/ValeriaPFR/Prueba_ObtencionPreparacionDatos_RTD-25-01-13-0058-4.git

Desarrollo

A. Introducción y Objetivo

El departamento de Recursos Humanos requería integrar y consolidar información dispersa sobre la dotación de personal y la estructura organizacional. El objetivo de esta evaluación es aplicar técnicas de combinación (merge), agrupación (groupby) y reestructuración matricial (pivoteo) utilizando la librería **Pandas**. Esto permite transformar variables atómicas en un reporte ejecutivo limpio que facilite la comparación directa de salarios promedio entre departamentos y sus ubicaciones geográficas.

B. Desarrollo e Implementación del Código

El desarrollo se estructuró en el script prueba_pandas.py. Se corrigieron las inconsistencias sintácticas de comillas presentes en el enunciado base para garantizar un flujo de ejecución continuo y libre de excepciones de compilación:

```
prueba_pandas.py 1, U X
prueba_pandas.py > ...
1  import pandas as pd
2  import numpy as np
3
4  # =====
5  # CONFIGURACIÓN DE LOS DATOS DE ORIGEN (Corrección de sintaxis del enunciado)
6  # =====
7
8  # DataFrame con información de empleados
9  data_empleados = {
10     'id_empleado': [101, 102, 103, 104, 105, 106, 107, 108],
11     'nombre': ['Ana', 'Luis', 'Sofia', 'Carlos', 'Elena', 'Pedro', 'Laura', 'David'],
12     'salario': [55000, 58000, 72000, 48000, 75000, 85000, 62000, 88000],
13     'id_departamento': ['V_01', 'V_01', 'T_01', 'M_01', 'T_01', 'D_01', 'M_01', 'D_01']
14 }
15 df_empleados = pd.DataFrame(data_empleados)
16
17 # DataFrame con información de departamentos
18 data_departamentos = {
19     'id_departamento': ['V_01', 'T_01', 'M_01', 'D_01'],
20     'nombre_depto': ['Ventas', 'Tecnología', 'Marketing', 'Dirección'],
21     'ubicacion': ['Norte', 'Sur', 'Norte', 'Norte']
22 }
23 df_departamentos = pd.DataFrame(data_departamentos)
24
25
26 # =====
27 # REQUERIMIENTO 1: Combinación de Datos (Merge)
28 # =====
29 print("=" * 65)
30 print("      REPORTE DE RECURSOS HUMANOS: ANÁLISIS SALARIAL      ")
31 print("=" * 65)
32
33 # Unimos los DataFrames usando como llave 'id_departamento' mediante un Left Join
34 df_completo = pd.merge(df_empleados, df_departamentos, on='id_departamento', how='left')
35
36 print("[REQUERIMIENTO 1] DataFrame Combinado ('df_completo'):")
37 print(df_completo.to_string(index=False))
38 print("-" * 65)
```

```

prueba_pandas.py 1, U X
prueba_pandas.py > ...
40
41 # =====
42 # REQUERIMIENTO 2: Agrupamiento y Agregación (Groupby)
43 # =====
44 print("\n[REQUERIMIENTO 2] Salario Promedio por Departamento y Ubicación:")
45
46 # Agrupamos por departamento y ubicación calculando la media del salario
47 df_agrupado = df_completo.groupby(['nombre_depto', 'ubicacion'])['salario'].mean().reset_index()
48
49 # Renombramos la columna resultante para mayor claridad
50 df_agrupado.rename(columns={'salario': 'salario_promedio'}, inplace=True)
51
52 print(df_agrupado.to_string(index=False))
53 print("-" * 65)
54
55
56 # =====
57 # REQUERIMIENTO 3: Pivoteo de Datos (Pivot Table)
58 # =====
59 print("\n[REQUERIMIENTO 3] Tabla Pivote Final para Comparación Directa:")
60
61 # Reestructuramos la información cruzando departamentos (índice) y ubicaciones (columnas)
62 df_pivote = df_agrupado.pivot(index='nombre_depto', columns='ubicacion', values='salario_promedio')
63
64 # Reemplazamos los valores nulos (NaN) por un guion para mejorar la presentación visual
65 df_pivote_presentacion = df_pivote.fillna('-')
66
67 print(df_pivote_presentacion)
68 print("-" * 65)

```

C. Resultados Obtenidos (Salida de la Consola)

Al ejecutar el script, la consola despliega los bloques de información ordenados cronológicamente:

```

PS C:\TD_EspecializacionCiencia de Datos\Nueva CIASPO\U2-ObtencionPreparacionDatos\U2-DesafiosEntregables\U3-ObtencionPreparacionDatos> python -u "c:\TD_Especializ
acionCiencia de Datos\Nueva CIASPO\U2-ObtencionPreparacionDatos\U2-DesafiosEntregables\U3-ObtencionPreparacionDatos\prueba_pandas.py"
=====
REPORTE DE RECURSOS HUMANOS: ANÁLISIS SALARIAL
=====
[REQUERIMIENTO 1] DataFrame Combinado ('df_completo'):
id_empleo nombre salario id_departamento nombre_depto ubicacion
101 Ana 55000 V_01 Ventas Norte
102 Luis 58000 V_01 Ventas Norte
103 Sofia 72000 T_01 Tecnología Sur
104 Carlos 48000 M_01 Marketing Norte
105 Elena 75000 T_01 Tecnología Sur
106 Pedro 85000 D_01 Dirección Norte
107 Laura 62000 M_01 Marketing Norte
108 David 88000 D_01 Dirección Norte
=====
[REQUERIMIENTO 2] Salario Promedio por Departamento y Ubicación:
nombre_depto ubicacion salario_promedio
Dirección Norte 86500.0
Marketing Norte 55000.0
Tecnología Sur 73500.0
Ventas Norte 56500.0
=====
[REQUERIMIENTO 3] Tabla Pivote Final para Comparación Directa:
ubicacion Norte Sur
nombre_depto
Dirección 86500.0 -
Marketing 55000.0 -
Tecnología - 73500.0
Ventas 56500.0 -
=====
PS C:\TD_EspecializacionCiencia de Datos\Nueva CIASPO\U2-ObtencionPreparacionDatos\U2-DesafiosEntregables\U3-ObtencionPreparacionDatos>

```

D. Conclusiones y Análisis Metodológico

- Consistencia en la Integración (Requerimiento 1): Mediante la función `pd.merge()` con estrategia Left Join, se garantizó la persistencia absoluta de los registros de la nómina de empleados. Cada trabajador quedó correctamente mapeado con sus metadatos organizacionales sin generar registros huérfanos.
- Reducción de Dimensiones (Requerimiento 2): La combinación de `.groupby()` junto con el estimador aritmético `.mean()` permitió colapsar los microdatos individuales en registros corporativos promedio. El uso de `.reset_index()` normalizó los niveles del DataFrame, preparando los datos para su posterior reestructuración.
- Simetría y Comparación Visual (Requerimiento 3): La implementación de `.pivot()` reorganizó las variables de estudio transponiendo la columna de ubicación geográfica en ejes horizontales independientes. Esta disposición optimiza la lectura analítica, evidenciando de forma inmediata las brechas o distribuciones salariales de la organización. Los valores no existentes en determinados cruces de categorías se manejaron mediante `.fillna('-')` para mantener una estética limpia y profesional.